Outputs from all algorithms have been attached in the zip file with the source code.

Q1A:

Pseudocode:

```
Public class quicksort():
array[ID = 1], arrayLength = 1000
array[ID = 2], arrayLength = 5000
... for all 7 days
randomiser():
        new random (1000 - 5000)
        return random
main():
        for ID (1 - 7):
        currentArray = array[key]
                for i (arrayLength, i++)
                        currentArray[i] = randomiser();
                current day = ""
                arrayLength = 0
                switch(key):
                        case 1:
                                currentDay = monday
                                arrayLength = 1000
                        case 2:
                                currentDay = tuesday
                                arrayLength = 5000
                        ... for all 7 days
                sortedCurrentArray = quickSort(currentArray, currentArrayLength - 1)
                print sortedCurrentArray
function pointer(arr, low, high):
   pivot = arr[high]
   i = low - 1
```

```
    for j from low to high - 1:
        if arr[j] <= pivot:
            i = i + 1
            swap arr[i] and arr[j]
    swap arr[i + 1] and arr[high]
    return i + 1

function quickSort(arr, low, high):
    if low < high:
        pi = pointer(arr, low, high)
        quickSort(arr, low, pi - 1)
        quickSort(arr, pi + 1, high)
```

Q1B:

Below is the run time from the efficiency test done in part 1B.

	array creation time	array sorting time
1		
Monday	2	0
Tuesday	1	0
Wednesday	1	0
Thursday	4	2
Friday	6	5
Saturday	7	5
Sunday	24	30
2		
Monday	2	0
Tuesday	0	0
Wednesday	1	1
Thursday	4	3
Friday	5	5
Saturday	6	5
Sunday	26	30
3		
Monday	2	1
Tuesday	0	2
Wednesday	1	0
Thursday	3	3
Friday	5	4
Saturday	6	6
Sunday	25	30
4		
Monday	2	0
Tuesday	0	1
Wednesday	1	1
Thursday	5	2
Friday	5	3
Saturday	7	5
Sunday	23	31
5		
Monday	1	1
Tuesday	0	1
Wednesday	2	0
Thursday	4	3
Friday	5	4
Saturday	6	5
Sunday	24	31
6		
Monday	2	0
Tuesday	1	0
Wednesday	1	0
Thursday	3	3
Friday	5	4
Saturday	6	6
Sunday	27	29
7		
Monday	2	0
Tuesday	0	0
Wednesday	1	0
Thursday	3	3
Friday	5	4
Saturday	7	4
Sunday	27	30
8		
Monday	1	1
Tuesday	0	1
Wednesday	0	1
Thursday	3	2
Friday	4	4
Saturday	6	5
Sunday	29	30
9		
Monday	2	0
Tuesday	0	1
Wednesday	1	1
Thursday	4	3
Friday	6	4
Saturday	6	5
Sunday	25	31
10		
Monday	2	0
Tuesday	0	0
Wednesday	2	1
Thursday	4	2
Friday	5	4
Saturday	5	6
Sunday	26	30

Q1C and Q1D:

This report details the creation of a hybrid sorting system by combining quicksort with insertion sort to enhance program efficiency. While quicksort excels with large datasets, its performance diminishes with smaller ones, prompting the need for expansion.

Quick sorts use a divide-and-conquer approach to break down the array. This means that this sort uses the time complexity O(n log n). When looking at O(n log n), you can see it starts off poor with few amounts of data but increases its speed as more data is entered into the array. Then, the time taken flattens out with a relatively consistency and only slightly increasing with more and more inputs. This makes it the perfect choice for our assignment as we are required to sort through thousands of pieces of data.

For the hybrid, we had the choice between an insertion sort or a selection sort. I initially decided that the best code to implement was an insertion sort, before later exploring the other possibilities available to us. I used a check for the length of the unsorted list within the already existing quicksort function (now named hybridQuickSort) to check whether the length of the unsorted list was greater than the set amount. Once it reached this threshold, it ran the aforementioned insertion sort. However, I then decided to see whether selection sort would have a better effect or whether making it until the list reaches a cut-off point would be better.

However, once I had made adjustments to the code with a selection sort, I was not able to notice any noticeable difference in the run time between the two of them, with a slight leniency towards the insertion sort. I ended up deciding that this was possibly the greatest option for a version of hybrid code available; hence, this is the one I stuck with.

When deciding which hybrid to go with, the main priority was efficiency. As for the time complexity for insertion and selection sorts, in both cases, the time complexity was $O(n^2)$. This means that as the list gets bigger, the time taken to sort only gets longer. This is great in combination with the quick sort method, as mentioned before, which uses O(n log n). In the end, I decided that the best option for the threshold should be 15. I decided this number by first trying 100, 50 and 10. 10 made little to no difference, while 100 and 50 increased the run time rather than decreasing it. Hence, the best parameter for this code was 15.

Overall, after testing and refining this code, the hybrid system demonstrated improvements in its run time in comparison to the standalone quick sort. By simply integrating a second sort into the code, I was able to refine my code and create a code capable of handing a variety of databases.